



Tribhuvan University

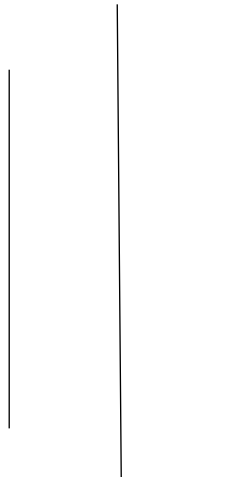
Faculty of Humanities and Social Sciences

Patan Multiple Campus

Master of Computer Application (MCA)

First Semester

Discrete Structure (MCA-501)



Submitted By:

Name: Sandesh Giri

Roll No: 2285076

Submitted To:

Department: Department of MCA

Instructor: Asst. Prof. Narayan Dhamala

Table of Contents

Lab-1: WAP in python to display truth table of negation.....	1
Lab 2: WAP in python to display truth table of conjunction.	2
Lab 3: WAP in python to display truth table of disjunction.	3
Lab 4: WAP in python to display truth table of implication.....	4
Lab 5: WAP in python to display truth table of bi-implication.	5
Lab 6: WAP in python to display truth table of bi-implication.	6
Lab 7: WAP in python to display truth table of bi-implication.	7
Lab 8: WAP in python to find the GCD between two input numbers using Extended Euclidean algorithm.	8
Lab 9: WAP in python to solve the using Chinese Remainder theorem.	9
Lab 10: WAP in python to test relation properties: Reflexive, Symmetric, Anti-symmetric and Transitive for a given set and Relation.	10
Lab 11: WAP in python to represent relation in matrix form for a given set and relation. .	11
Lab 12: WAP in python to represent relation in digraph form for a given set and relation.	12
Lab 13: WAP in python to determine whether the given relation is equivalence relation or not.	13
Lab 14: WAP in python to determine whether the given relation is partial order relation or not.	14
Lab 15: WAP in python to find the factorial of an input number using recursion.	15
Lab 16: WAP in python to find the nth term of Fibonacci sequence using recursion.	16
Lab 17: WAP in python to validate inductive proof.....	17
Lab 18: WAP in python to represent adjacency list and visualize using NetworkX.	18
Lab 19: WAP In python to represent adjacency matrix and visualize using NetworkX.	19
Lab 20: WAP in python to perform tree traversal (pre-order, in-order and post-order).....	20
Lab 21: WAP in python to find MST using Kruskal's algorithm.....	21
Lab 22: WAP in python to construct finite automata that accepts strings only ab and abb.	22

Lab-1: WAP in python to display truth table of negation.

Negation is a unary logical operator that reverses the truth value of a proposition. If proposition P is true, its negation ($\neg P$) is false, and vice versa.

p	$\neg p$
T	F
F	T

Source Code:

```
print("--- Truth Table for Negation (NOT) ---")
print("P\t\t\t~P")
print("-" * 20)
for p in [True, False]:
    print(f"{int(p)}\t\t\t{int(not p)}")
```

```
C:\Users\sande\PycharmProjects\SandeshGiri\
--- Truth Table for Negation (NOT) ---
P   |   ~P
-----
1   |   0
0   |   1

Process finished with exit code 0
```

In this experiment, we successfully implemented a Python program to generate and display the truth table for logical negation, verifying that the output is always the opposite truth value of the input proposition.

Lab 2: WAP in python to display truth table of conjunction.

Conjunction ($\wedge$ or AND) is a binary logical operation that is true only if both operands are true; otherwise, it is false.

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

```
print("--- Truth Table for Conjunction (AND) ---")
print("P\tQ\t\tP and Q")
print("-" * 30)
for p in [True, False]:
    for q in [True, False]:
        print(f'{int(p)}\t{int(q)}\t\t{int(p and q)}')
```

```
C:\Users\sande\PycharmProjects\SandeshGiri
--- Truth Table for Conjunction (AND) ---
P   Q   |   P and Q
-----
1   1   |   1
1   0   |   0
0   1   |   0
0   0   |   0

Process finished with exit code 0
```

The program successfully generated the truth table of conjunction using the Python and operator.

Lab 3: WAP in python to display truth table of disjunction.

Disjunction ($\vee$ or OR) is a binary logical operation that is true if at least one of the operands is true, and false only when both operands are false.

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

```
print("--- Truth Table for Disjunction (OR) ---")
print("P\tQ\t\tP or Q")
print("-" * 30)
for p in [True, False]:
    for q in [True, False]:
        print(f'{int(p)}\t{int(q)}\t\t{int(p or q)}')
```

```
C:\Users\sande\PycharmProjects\SandeshGiri\
--- Truth Table for Disjunction (OR) ---
P   Q   |   P or Q
-----
1   1   |   1
1   0   |   1
0   1   |   1
0   0   |   0

Process finished with exit code 0
```

The program successfully displayed the truth table of disjunction using the Python or operator.

Lab 4: WAP in python to display truth table of implication.

Implication ($\rightarrow$ or Conditional) $P \rightarrow Q$ is false only when the premise P is true and the conclusion Q is false; in all other cases, it is true. In Python, $P \rightarrow Q$ is equivalent to $(\text{not } P) \vee Q$.

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

```
print("--- Truth Table for Implication (P -> Q) ---")
print("P\tQ\t\tP -> Q")
print("-" * 30)
for p in [True, False]:
    for q in [True, False]:
        impl = (not p) or q
        print(f'{int(p)}\t{int(q)}\t\t{int(impl)}')
```

```
C:\Users\sande\PycharmProjects\SandeshGiri\
--- Truth Table for Implication (P -> Q) ---
P   Q   |   P -> Q
-----
1   1   |   1
1   0   |   0
0   1   |   1
0   0   |   1

Process finished with exit code 0
```

The program successfully generated the truth table of implication using the formula $\neg P \vee Q$.

Lab 5: WAP in python to display truth table of bi-implication.

Bi-implication ($\leftrightarrow$ or Biconditional) $P \leftrightarrow Q$ is true if and only if both propositions have the same truth value (both true or both false).

p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

```
print("--- Truth Table for Bi-implication (P <-> Q) ---")
print("P\tQ\tP <-> Q")
print("-" * 30)
for p in [True, False]:
    for q in [True, False]:
        bi_impl = (p == q)
        print(f'{int(p)}\t{int(q)}\t{int(bi_impl)}')
```

```
C:\Users\sande\PycharmProjects\SandeshGiri\.venv\!
--- Truth Table for Bi-implication (P <-> Q) ---
P   Q   |   P <-> Q
-----
1   1   |   1
1   0   |   0
0   1   |   0
0   0   |   1

Process finished with exit code 0
```

The program successfully generated the truth table of bi-implication by comparing the truth values of two propositions.

Lab 6: WAP in python to display truth table of bi-implication.

A tautology is a compound proposition that is unconditionally true for all possible truth values of its propositional variables. For example, $P \vee \neg P$ is a tautology.

p	¬p	p ∨ ¬p
T	F	T
F	T	T

```
print("Verification of Tautology")
print(f'{'P':<10}{'NOT P':<10}{'P OR NOT P':<15}')
print("-" * 35)
tautology = True
for p in [True, False]:
    not_p = not p
    result = p or not_p
    print(f'{'str(p):<10}{'str(not_p):<10}{'str(result):<15}')
```

```
    if not result:
        tautology = False
print("-" * 35)
if tautology:
    print("The proposition is a tautology.")
else:
    print("The proposition is not a tautology.")
```

```
C:\Users\sande\PycharmProjects\SandeshGiri\
Verification of Tautology
P          NOT P      P OR NOT P
-----
True       False      True
False      True        True
-----
The proposition is a tautology.
```

The program verified the given logical proposition and determined whether it is a tautology by checking all possible truth values.

Lab 7: WAP in python to display truth table of bi-implication.

The Euclidean algorithm computes the greatest common divisor (GCD) of two integers a and b efficiently by repeatedly applying division remainder: $\text{gcd}(a, b) = \text{gcd}(b, a \text{ (mod } b))$ until $b = 0$.

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))
```

```
x = a  
y = b
```

```
while y != 0:  
    x, y = y, x % y
```

```
print("GCD =", abs(x))
```

```
C:\Users\sande\PycharmProjects\SandeshGiri\  
Enter first number: 29  
Enter second number: 12  
GCD = 1  
  
Process finished with exit code 0  
.
```

The program used repeated division with remainder to compute the GCD efficiently using the Euclidean algorithm.

Lab 8: WAP in python to find the GCD between two input numbers using Extended Euclidean algorithm.

The Extended Euclidean Algorithm not only finds the GCD of two numbers but also finds integers x and y satisfying: $ax + by = \text{gcd}(a,b)$. These values are called Bézout coefficients.

```
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    gcd, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
    return gcd, x, y

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
gcd, x, y = extended_gcd(a, b)
print("GCD =", gcd)
print("x =", x)
print("y =", y)
print("Verification =", a * x + b * y)
```

```
C:\Users\sande\PycharmProjects\SandeshGiri\
Enter first number: 22
Enter second number: 15
GCD = 1
x = -2
y = 3
Verification = 1

Process finished with exit code 0
```

The program found both the GCD and Bézout coefficients using the Extended Euclidean algorithm and verified the identity.

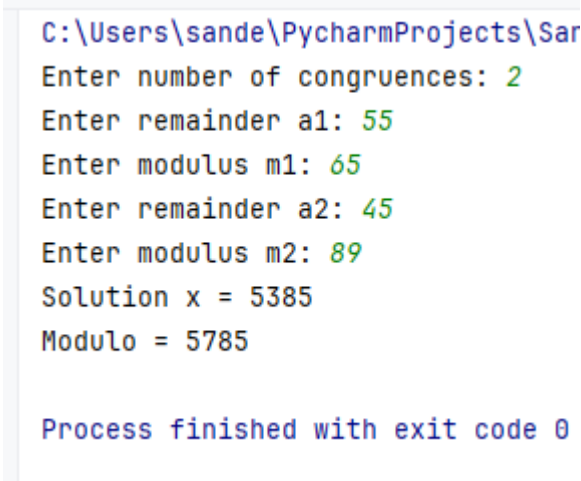
Lab 9: WAP in python to solve the given simultaneous congruence using Chinese Remainder theorem.

The Chinese Remainder Theorem (CRT) gives a unique solution modulo the product of pairwise coprime moduli for a system of simultaneous congruences.

For $x \equiv a_i \pmod{n_i}$, let $N = \prod n_i$, $N_i = N/n_i$, and $M_i = N_i^{-1} \pmod{n_i}$. Then $x \equiv \sum(a_i N_i M_i) \pmod{N}$.

```
from math import prod
n = int(input("Enter number of congruences: "))
a = []
m = []

for i in range(n):
    a.append(int(input(f'Enter remainder a{i + 1}: ')))
    m.append(int(input(f'Enter modulus m{i + 1}: ')))
M = prod(m)
x = 0
for i in range(n):
    Mi = M // m[i]
    for j in range(1, m[i]):
        if (Mi * j) % m[i] == 1:
            inverse = j
            break
    x += a[i] * Mi * inverse
x = x % M
print("Solution x =", x)
print("Modulo =", M)
```



```
C:\Users\sande\PycharmProjects\Sar
Enter number of congruences: 2
Enter remainder a1: 55
Enter modulus m1: 65
Enter remainder a2: 45
Enter modulus m2: 89
Solution x = 5385
Modulo = 5785

Process finished with exit code 0
```

The program applied the Chinese Remainder Theorem and obtained the simultaneous solution $x = 23$ modulo 105 for the sample system.

Lab 10: WAP in python to test relation properties: Reflexive, Symmetric, Anti-symmetric and Transitive for a given set and Relation.

A binary relation R on a set A can be tested for reflexivity, symmetry, anti-symmetry, and transitivity by checking the ordered pairs required by each definition.

Reflexive: $(a,a) \in R$. Symmetric: $(a,b) \in R \Rightarrow (b,a) \in R$. Anti-symmetric: $(a,b), (b,a) \in R \Rightarrow a=b$. Transitive: $(a,b), (b,c) \in R \Rightarrow (a,c) \in R$.

```
A = set(map(int, input("Enter elements of set: ").split()))
n = int(input("Enter number of relation pairs: "))
R = set()
for i in range(n):
    a, b = map(int, input("Enter pair: ").split())
    R.add((a, b))
reflexive = all((a, a) in R for a in A)
symmetric = all((b, a) in R for a, b in R)
antisymmetric = all(
    not ((b, a) in R and a != b)
    for a, b in R
)
transitive = all(
    (a, c) in R
    for a, b in R
    for c in A
    if (b, c) in R
)
print("Reflexive:", reflexive)
print("Symmetric:", symmetric)
print("Anti-symmetric:", antisymmetric)
print("Transitive:", transitive)
```

Output:

```
C:\Users\sande\PycharmProjects\SandeshGiri\
Enter elements of set: 1 2 3
Enter number of relation pairs: 2
Enter pair: 1 1
Enter pair: 2 2
Reflexive: False
Symmetric: True
Anti-symmetric: True
Transitive: True

Process finished with exit code 0
|
```

Conclusion:

The program checked each relation property directly from its mathematical definition and reported the result for the given relation.

Lab 11: WAP in python to represent relation in matrix form for a given set and relation.

A relation on a finite set can be represented by a 0–1 matrix. Entry $M[i][j]$ is 1 when (a_i, a_j) belongs to the relation, otherwise it is 0.

$M_{ij} = 1$ if $(a_i, a_j) \in R$; otherwise $M_{ij} = 0$.

```
A = list(map(int, input("Enter elements of set: ").split()))
```

```
n = int(input("Enter number of relation pairs: "))
```

```
R = set()
```

```
for i in range(n):
```

```
    a, b = map(int, input("Enter pair: ").split())
```

```
    R.add((a, b))
```

```
print("\nRelation Matrix:")
```

```
for a in A:
```

```
    for b in A:
```

```
        if (a, b) in R:
```

```
            print(1, end=" ")
```

```
        else:
```

```
            print(0, end=" ")
```

```
    print()
```

Output:

```
C:\Users\sande\PycharmProjects\SandeshGiri\  
Enter elements of set: 1 2 3  
Enter number of relation pairs: 2  
Enter pair: 1 1  
Enter pair: 2 2  
  
Relation Matrix:  
1 0 0  
0 1 0  
0 0 0  
  
Process finished with exit code 0
```

Conclusion:

The relation was successfully converted into a binary matrix by marking present ordered pairs with 1 and absent pairs with 0.

Lab 12: WAP in python to represent relation in digraph form for a given set and relation.

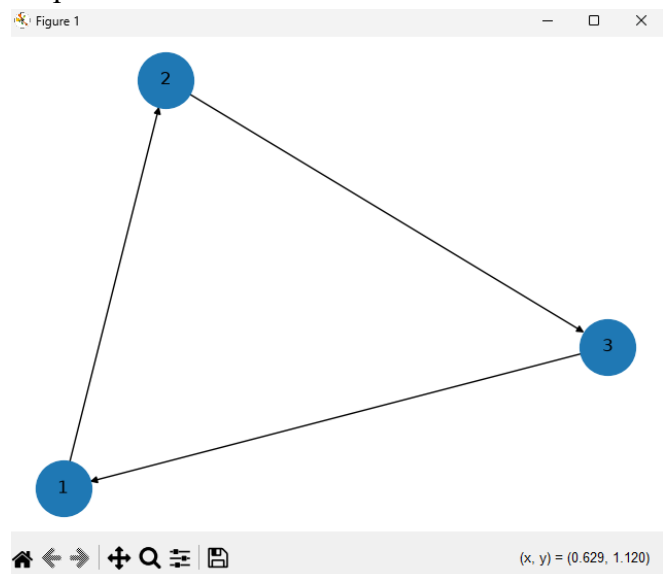
A relation can be represented as a directed graph (digraph), where each set element is a vertex and every ordered pair (a,b) is drawn as a directed edge $a \rightarrow b$.

Vertices = elements of A; directed edges = ordered pairs in R.

```
import networkx as nx
import matplotlib.pyplot as plt
A = list(map(int, input("Enter elements of set: ").split()))
n = int(input("Enter number of relation pairs: "))
R = []
```

```
for i in range(n):
    a, b = map(int, input("Enter pair: ").split())
    R.append((a, b))
G = nx.DiGraph()
G.add_nodes_from(A)
G.add_edges_from(R)
nx.draw(
    G,
    with_labels=True,
    node_size=1500,
    arrows=True
)
plt.title("Directed Graph of Relation")
plt.show()
```

Output:



Conclusion:

The program represented the given relation as a directed graph using NetworkX, with ordered pairs shown as directed edges.

Lab 13: WAP in python to determine whether the given relation is equivalence relation or not.

A relation is an equivalence relation if and only if it is reflexive, symmetric, and transitive.

Equivalence relation $\Leftrightarrow$ Reflexive $\wedge$ Symmetric $\wedge$ Transitive.

```
A = set(map(int, input("Enter elements of set: ").split()))
```

```
n = int(input("Enter number of relation pairs: "))
```

```
R = set()
```

```
for i in range(n):
```

```
    a, b = map(int, input("Enter pair: ").split())
```

```
    R.add((a, b))
```

```
reflexive = all((a, a) in R for a in A)
```

```
symmetric = all((b, a) in R for a, b in R)
```

```
transitive = all(
```

```
    (a, c) in R
```

```
    for a, b in R
```

```
    for c in A
```

```
    if (b, c) in R
```

```
)
```

```
if reflexive and symmetric and transitive:
```

```
    print("The relation is an equivalence relation.")
```

```
else:
```

```
    print("The relation is not an equivalence relation.")
```

Output:

```
C:\Users\sande\PycharmProjects\SandeshGiri\.ver
Enter elements of set: 1 2 3
Enter number of relation pairs: 3
Enter pair: 1 3
Enter pair: 1 2
Enter pair: 2 3
The relation is not an equivalence relation.
```

Conclusion:

The program tested reflexivity, symmetry, and transitivity and determined that the sample relation satisfies all conditions for equivalence.

Lab 14: WAP in python to determine whether the given relation is partial order relation or not.

A relation is a partial order if it is reflexive, anti-symmetric, and transitive. A set together with a partial order forms a partially ordered set (poset).

Partial order $\Leftrightarrow$ Reflexive $\wedge$ Anti-symmetric $\wedge$ Transitive

```
A = set(map(int, input("Enter elements of set: ").split()))
n = int(input("Enter number of relation pairs: "))
R = set()
for i in range(n):
    a, b = map(int, input("Enter pair: ").split())
    R.add((a, b))
reflexive = all((a, a) in R for a in A)
antisymmetric = all(
    not ((b, a) in R and a != b)
    for a, b in R
)
transitive = all(
    (a, c) in R
    for a, b in R
    for c in A
    if (b, c) in R
)
if reflexive and antisymmetric and transitive:
    print("The relation is a partial order relation.")
else:
    print("The relation is not a partial order relation.")
```

Output:

```
C:\Users\sande\PycharmProjects\SandeshGiri\.venv
Enter elements of set: 1 2 3
Enter number of relation pairs: 3
Enter pair: 1 2
Enter pair: 2 3
Enter pair: 3 1
The relation is not a partial order relation.
```

Conclusion

The program verified that the sample relation is reflexive, anti-symmetric, and transitive, so it is a partial order.

Lab 15: WAP in python to find the factorial of an input number using recursion.

Factorial of a non-negative integer n is the product of all positive integers from 1 to n . A recursive definition reduces $n!$ to $n \times (n-1)!$ until the base case is reached.

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
n = int(input("Enter a number: "))

print("Factorial =", factorial(n))
```

Output:

```
C:\Users\sande\PycharmProjects\SandeshGiri\
Enter a number: 10
Factorial = 3628800
```

Conclusion:

The factorial was calculated recursively by reducing the problem to smaller factorials until the base case $0!$ or $1!$ was reached.

Lab 16: WAP in python to find the nth term of Fibonacci sequence using recursion.

The Fibonacci sequence begins with 0 and 1. Every later term is the sum of the two preceding terms. The same recurrence can be implemented directly using recursion.

```
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)
n = int(input("Enter the term number: "))

print("Fibonacci term =", fibonacci(n))
```

Output:

```
C:\Users\sande\PycharmProjects\SandeshGiri\
Enter the term number: 15
Fibonacci term = 610
```

Conclusion:

The program found the nth Fibonacci term recursively by applying the Fibonacci recurrence relation until reaching the base cases.

Lab 17: WAP in python to validate inductive proof.

Mathematical induction proves a statement for all natural numbers by checking a base case and then proving that truth for k implies truth for $k+1$. This program computationally validates the identity $1+2+\dots+n = n(n+1)/2$ for a selected range.

```
n = int(input("Enter maximum value of n: "))

valid = True

for i in range(1, n + 1):
    left = sum(range(1, i + 1))
    right = i * (i + 1) // 2

    print("n =", i, "LHS =", left, "RHS =", right)

    if left != right:
        valid = False

if valid:
    print("\nThe formula is verified for all tested values.")
else:
    print("\nThe formula is not verified.")
```

Output:

```
C:\Users\sande\PycharmProjects\SandeshGiri\.venv
Enter maximum value of n: 5
n = 1 LHS = 1 RHS = 1
n = 2 LHS = 3 RHS = 3
n = 3 LHS = 6 RHS = 6
n = 4 LHS = 10 RHS = 10
n = 5 LHS = 15 RHS = 15

The formula is verified for all tested values.
```

Conclusion:

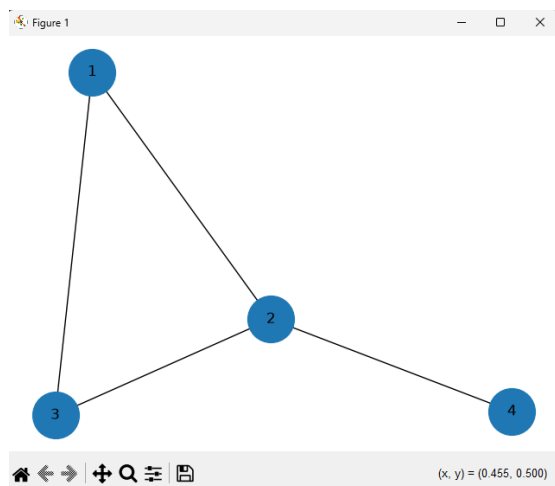
The program validated the base case and inductive progression of the sum-of-natural-numbers identity for the chosen range, illustrating the induction process.

Lab 18: WAP in python to represent adjacency list and visualize using NetworkX.

An adjacency list stores, for each graph vertex, the vertices directly connected to it. It is memory-efficient for sparse graphs and can be visualized using NetworkX.

```
import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
edges = [
    (1, 2),
    (1, 3),
    (2, 3),
    (2, 4)
]
G.add_edges_from(edges)
print("Adjacency List:")
for node in G.nodes():
    print(node, ":", list(G.neighbors(node)))
nx.draw(
    G,
    with_labels=True,
    node_size=1500
)
plt.title("Graph using Adjacency List")
plt.show()
```

Output:



Conclusion:

The program printed the adjacency list of the graph and visualized the same graph using NetworkX and Matplotlib.

Lab 19: WAP In python to represent adjacency matrix and visualize using NetworkX.

An adjacency matrix is a square matrix where entry $A[i][j]$ indicates whether an edge exists between vertices i and j . For an unweighted graph, 1 represents an edge and 0 represents no edge.

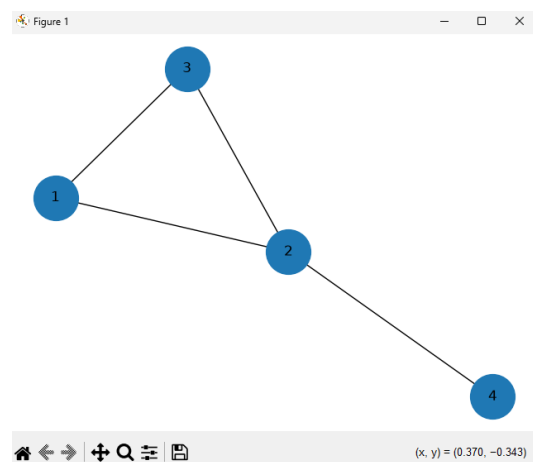
```
Conimport networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
edges = [
    (1, 2),
    (1, 3),
    (2, 3),
    (2, 4)
]
G.add_edges_from(edges)
nodes = sorted(G.nodes())
print("Adjacency Matrix:")
matrix = nx.to_numpy_array(G, nodelist=nodes, dtype=int)

for row in matrix:
    print(*row)

nx.draw(
    G,
    with_labels=True,
    node_size=1500
)
plt.title("Graph using Adjacency Matrix")
plt.show()
```

Output:

```
C:\Users\sande\Pyche
Adjacency Matrix:
0 1 1 0
1 0 1 1
1 1 0 0
0 1 0 0
```



Conclusion:

The program generated the adjacency matrix and displayed a NetworkX visualization of the corresponding graph.

Lab 20: WAP in python to perform tree traversal (pre-order, in-order and post-order).

Tree traversal visits every node of a tree in a defined order. In a binary tree, pre-order visits Root–Left–Right, in-order visits Left–Root–Right, and post-order visits Left–Right–Root.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)

print("Pre-order:")
preorder(root)
print("\nIn-order:")
inorder(root)
print("\nPost-order:")
postorder(root)
```

Output:

```
C:\Users\sande\PycharmProjects\SandeshGiri
Pre-order:
1 2 4 5 3
In-order:
4 2 5 1 3 |
Post-order:
4 5 2 3 1
```

Conclusion:

The program traversed the binary tree successfully using recursive pre-order, in-order, and post-order traversal techniques.

Lab 21: WAP in python to find MST using Kruskal's algorithm.

Kruskal's algorithm finds a Minimum Spanning Tree (MST) of a connected weighted undirected graph. It repeatedly selects the smallest-weight edge that does not create a cycle.

```
class DisjointSet:
    def __init__(self, vertices):
        self.parent = {v: v for v in vertices}
    def find(self, v):
        if self.parent[v] != v:
            self.parent[v] = self.find(self.parent[v])
        return self.parent[v]
    def union(self, a, b):
        root_a = self.find(a)
        root_b = self.find(b)
        if root_a != root_b:
            self.parent[root_b] = root_a
        return True
    return False
```

```
vertices = {1, 2, 3, 4}
```

```
edges = [
    (1, 2, 10),
    (1, 3, 6),
    (1, 4, 5),
    (2, 4, 15),
    (3, 4, 4)
]
```

```
edges.sort(key=lambda x: x[2])
```

```
ds = DisjointSet(vertices)
```

```
mst = []
```

```
total_weight = 0
```

```
for u, v, weight in edges:
```

```
    if ds.union(u, v):
```

```
        mst.append((u, v, weight))
```

```
        total_weight += weight
```

```
print("Edges in Minimum Spanning Tree:")
```

```
for u, v, weight in mst:
```

```
    print(u, "-", v, ":", weight)
```

```
print("Total weight =", total_weight)
```

Output:

```
C:\Users\sande\PycharmProjects\s
Edges in Minimum Spanning Tree:
3 - 4 : 4
1 - 4 : 5
1 - 2 : 10
Total weight = 19
```

Conclusion:

The program traversed the binary tree successfully using recursive pre-order, in-order, and post-order traversal techniques.

Lab 22: WAP in python to construct finite automata that accepts strings only ab and abb.

A finite automaton consists of a finite set of states, an input alphabet, a transition function, a start state, and one or more accepting states. The required automaton accepts exactly the strings "ab" and "abb".

```
def accepts(string):
    state = "q0"
    for symbol in string:
        if state == "q0" and symbol == "a":
            state = "q1"
        elif state == "q1" and symbol == "b":
            state = "q2"
        elif state == "q2" and symbol == "b":
            state = "q3"
        else:
            return False
    return state in ["q2", "q3"]

string = input("Enter a string: ")

if accepts(string):
    print("String Accepted")
else:
    print("String Rejected")
```

Output:

```
C:\Users\sande\PycharmF
Enter a string: ab
String Accepted
```

Conclusion:

The program traversed the binary tree successfully using recursive pre-order, in-order, and post-order traversal techniques.